Ansible

Why, What and How ;)

Brice Ekane
brice.ekane@example.org
https://github.com/tlc-2025/tlc.git
2025-2026

with some materials from Philippe merle, Théo Giraudet, Jamie Duncan, and Avinash Pawar

Talk outline

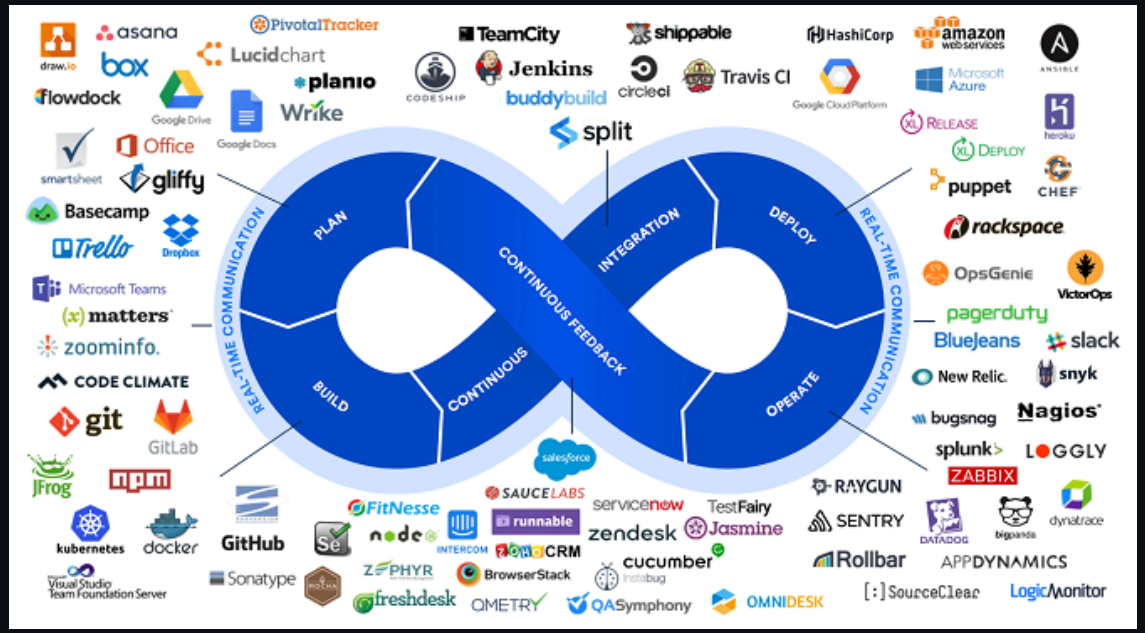
1. Why ?
    - Automation ?
    - Why Infrastructure as code ?
2. What ?
    - Ansible main concepts
3. How
    - Best practices
4. Related work

Why (IAC) ?

AUTOMATION == DOCUMENTATION

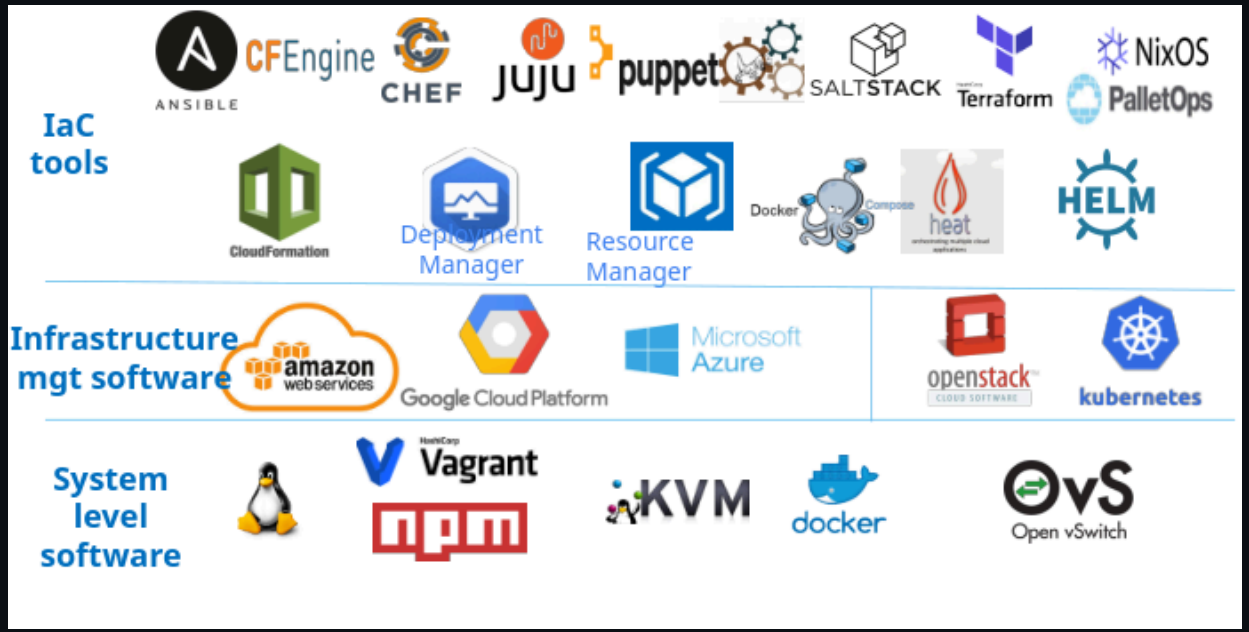
If done properly, the process of automating a process can become the documentation for the process.
Everything in Ansible (and most IAC tool) revolves around this core concept.

The DevOps Jungle

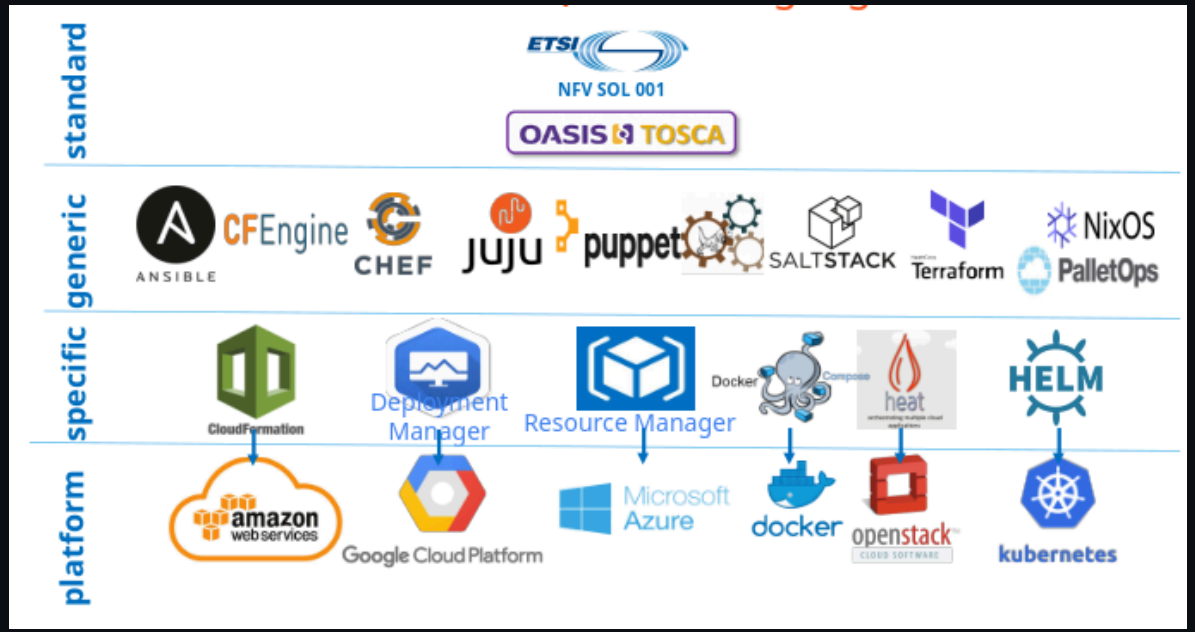


The jungle of Infrastructure as Code

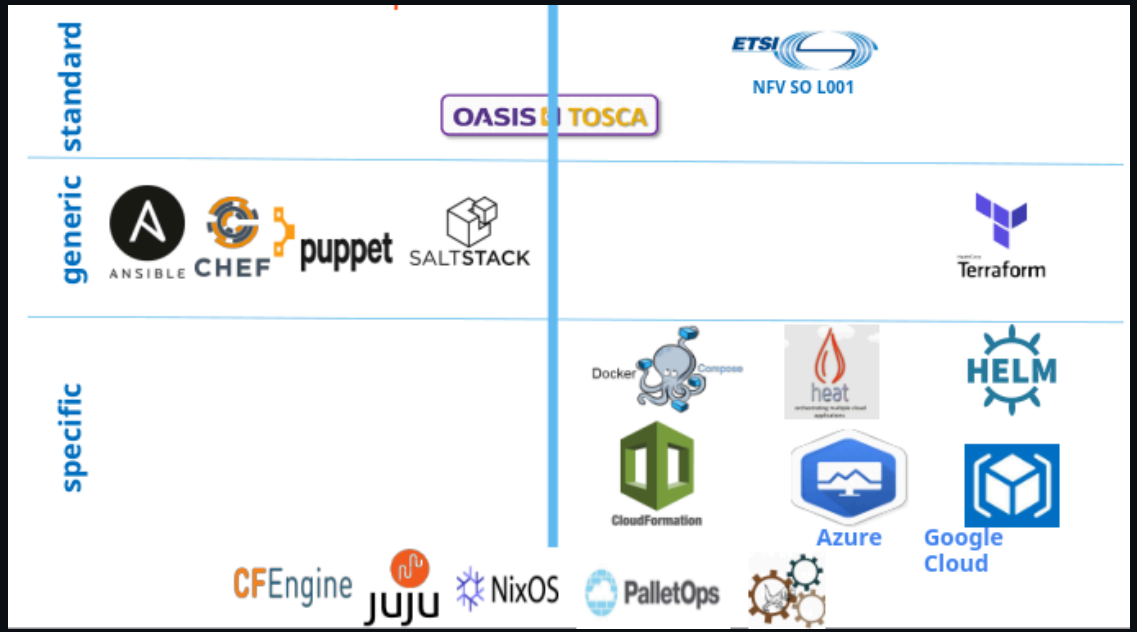
A taxonomy of the Infrastructure as Code



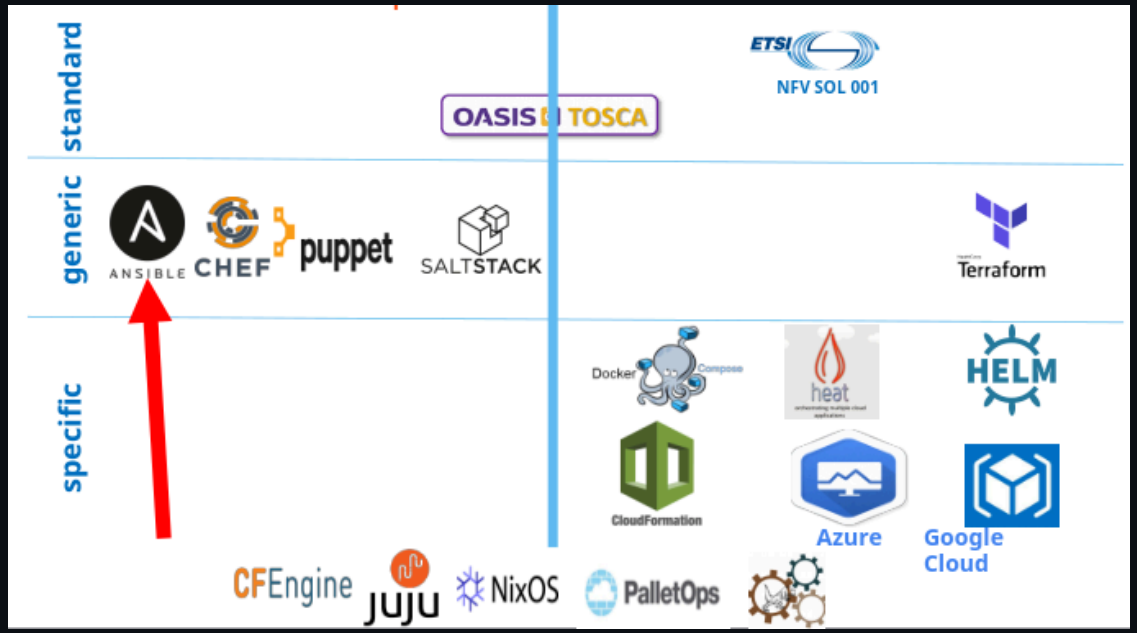
A taxonomy of IaC languages



Imperative versus Functional



Imperative versus Functional



Ansible main concepts

Agenda

1. Ansible Introduction
2. Why Ansible?
3. Ansible Architecture / How Ansible Works?
4. Ansible Components/Concepts
5. Ansible – Playbooks
6. Ansible – Role
7. Demo

Introduction - What is Ansible?

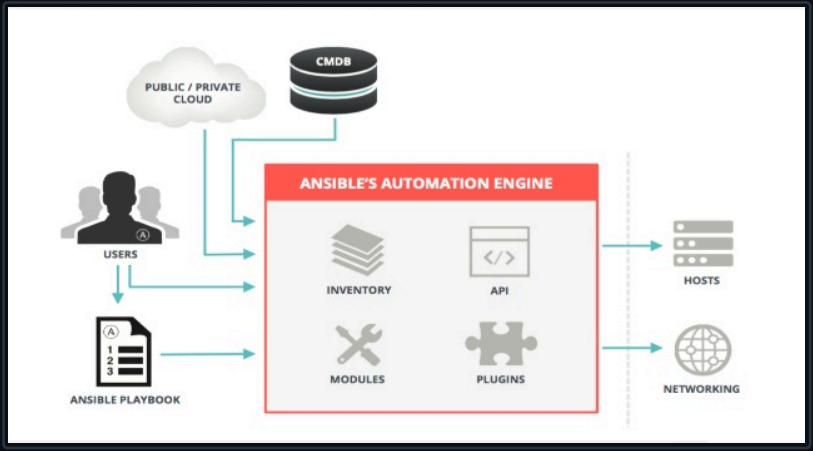
“Simple, agent-less and powerful open source IT automation Tool”

Why Ansible

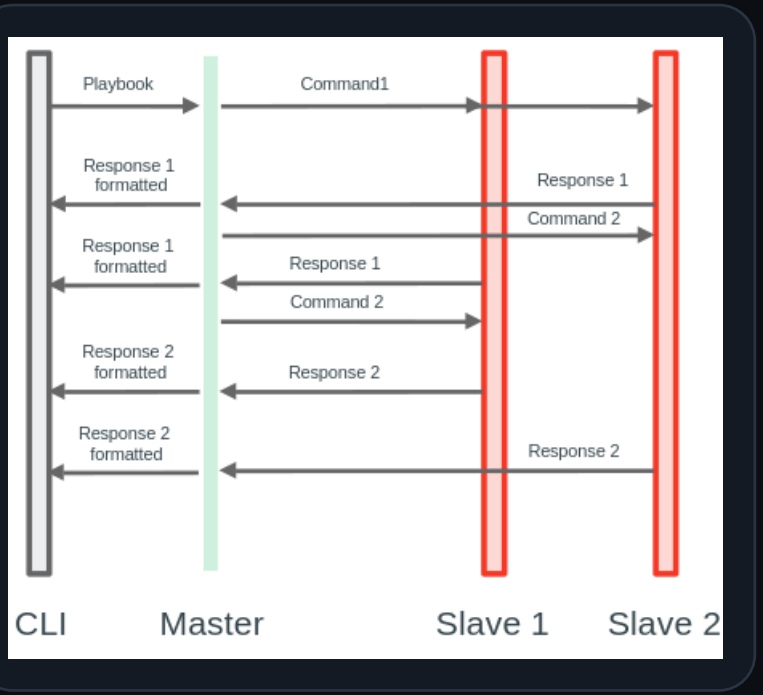
- It is very simple and powerful
- Agent-less – No need for agent installation and management
- Secure – SSH based connection
- Python / yaml based
- Highly flexible and configuration management of systems.
- Large number of ready to use modules available for system management
- Custom modules can be added if needed
- Human readable & Self documenting
- Idempotent

How Ansible Works?

Ansible works by connecting to your nodes and pushing out small programs, called “Ansible Modules” to them. Ansible then executes these modules (over SSH by default) and removes then when finished



How Ansible Works?



- Send commands via SSH
- Requires installation on the master side only
- Encrypted exchange
- May require Python on slave side (which can be installed by Ansible)
- Supported by RedHat
- Requires a key-secured SSH connection

Ansible installation

Activité

```shell
sudo apt-get install ansible
sudo dnf install ansible
sudo apk add ansible
...
```

Ansible main Concepts

- Inventory

  File where you declare the list of hosts and groups
- Modules

  Ansible ships with a number of modules (called the ‘module library’) that can be executed directly on remote hosts or through Playbooks. Users can also write their own modules
- Playbooks – Play, Tasks and Modules

  A playbook is a yaml file where you run series of tasks on the hosts
- Variables

  Ansible provides a mechanism for overriding variables.
- Templates

  Templates are a powerful resource for generating files on the hosts. A template will have a common structure and it will be populated with specify variable values at runtime.
- Roles Roles are ways of automatically loading certain vars_files, tasks, templates, and handlers based on a known file structure. Grouping content by roles also allows easy sharing of roles with other users.

Ansible - Hosts

Hosts : File containing the IPs/domain names of the remote machines on which to run the playbooks, in YML or INI format

```ini
[servers]
   server1
   server2
[bdd]
  10.10.10.2
```

(1) Name of the group

(2) Hosts/Nodes

It is possible to restrict the execution of a state to a certain group

Inventory / Host file

```ini
bekaneap@ptb-12g0s54:~/dev/TLC/tpl$ cat hosts
mail.example.com
[webservers]
foo.example.com
bar.example.com

[dbservers]
one.example.com
two.example.com
three.example.com
bekaneap@ptb-12g0s54:~/dev/TLC/tpl$
```

(1) Group

(2) Hosts/Nodes

Ansible - Playbook

```yaml
- name: installation
  become: yes
  apt:
    name: nginx
    state: latest
- name: start
  become: yes
  service:
    name: nginx
    state: started
```

(1) required sudo

(2) module name

(3) service information

Playbook -Strcuture

```yml
---
- name : Sample demo
  hosts: webserver
  become: true
  roles:
    - {role: apache}
```

(1) Play

(2) Role

Roles Directorie Structure

```
bekaneap@ptb-12g0s54:~/dev/TLC/tpl/toto$ tree
my-role/
my-role/
├── defaults
│   └── main.yml
├── files
├── handlers
│   └── main.yml
├── meta
│   └── main.yml
├── README.md
├── tasks
│   └── main.yml
├── templates
├── tests
│   ├── inventory
│   └── test.yml
└── vars
    └── main.yml

9 directories, 8 files
bekaneap@ptb-12g0s54:~/dev/TLC/tpl/toto$
```

Ansible - Execution

```
ansible-playbook -i hosts.ini foo.yml
```

`-- become`

Specify that all states must be applied in sudo

`-u <user>`

Specify the user on the host machine

`-K`

Ask for a password entry for the sudo

Ansible - Execution

https://github.com/barais/demoAnsible

Use vagrant to create VMs on top of Virtual Box / libivirt (Qemu/Kvm)

Demo1: deploy a simple lamp with only one playbook on one server

Demo2: deploy a simple lamp with two nodes (1 webserver, 1db) with a playbook that contains several roles. Deploy a simple php app to check the connection between php and mysql.

Ansible

Best practices

WORKFLOW

Treat Ansible content like application code
Version control is your best friend

Start as simple as possible and iterate
- Start with a basic playbook and static inventory
- Refactor and modularize progressively as you and your environment mature

Workflow

Do It with Style

- Create a style guide for all contributors
- Consistency in:
  - Tagging
  - Whitespace
  - Naming of Tasks, Plays, Variables, and Roles
  - Directory Layouts
- Enforce the style

PROJECT LAYOUTS: BASIC

```shell
bekaneap@ptb-12g0s54:~/dev/TLC/tpl$ tree basic-project/
basic-project/
├── group_vars
├── host_vars
│   └── hosts
├── inventory
└── site.yml
```

PROJECT LAYOUTS: ORGANIZATIONAL ROLES

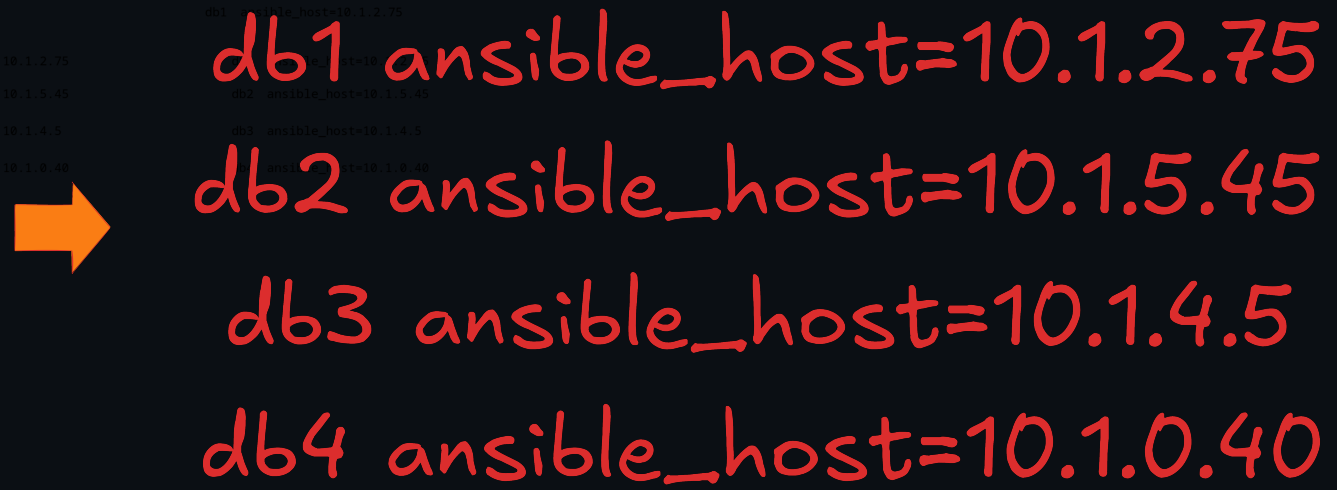
```
myapp/
├── roles
│   ├── myapp
│   │   ├── tasks
│   │   │   └── main.yml
│   │   ├── templates
│   ├── nginx
│   │   ├── tasks
│   │   │   └── main.yml
│   │   ├── templates
│   └── proxy
│       ├── tasks
│       │   └── main.yml
│       └── vars
│           └── main.yml
└── site.yml
```

PROJECT LAYOUTS: SHARED ROLES

```shell
myapp/
├── config.yml
├── provision.yml
├── roles
│   └── requirements.yml
└── site.yml
```

INVENTORY

Give inventory nodes human-meaningful names rather than IPs or DNS hostnames.



INVENTORY

w14301.acme.com
w17802.acme.com
w19203.acme.com
w19304.acme.com

web1 ansible_host=w14301.acme.com
web2 ansible_host=w17802.acme.com
web3 ansible_host=w19203.acme.com
web4 ansible_host=w19203.acme.com

INVENTORY

WHAT

```ini
[db]
 db[1:4]
[web]
 web[1:4]
```

WHERE

```ini
[east]
  db1
  web1
  db3
  web3
[west]
  db2
  web2
  db4
  web4
```

WHEN

```ini
[dev]
db1
web1
[test]
db2
web2
[prod]
db3
web3
db4
web4
```

INVENTORY

Use a single source of truth if you have it – even if you have multiple sources, Ansible can unify them.
Stay in sync automatically Reduce human error
Use your instance and provider metadata for more than pretty columns in your TPS reports

VARIABLES

Proper variable naming can make plays more readable and avoid variable name conflicts Use descriptive, unique human-meaningful variable names

Prefix variables with it’s “owner” such as a role name, service, or package

```Shell
apache_max_keepalive: 25
apache_port: 80
tomcat_port: 8080
```

VARIABLES

Make the most of variables

Find the appropriate place for your variables based on what, where and when they are set or modified

Separate logic (tasks) from variables to reduce repetitive patterns and provided added flexibility.

SEPARATE LOGIC FROM VARIABLES

```yml
- name: Clone student lesson app for a user host:
nodes
tasks:
- name: Create ssh dir
  file:
  state: directory
  path: /home/{{ username }}/.ssh
- name: Set Deployment Key
  copy:
  src: files/deploy_key
  dest: /home/{{ username }}/.ssh/id_rsa
- name: Clone repo
  git:
  accept_hostkey: yes
  clone: yes
  dest: /home/{{ username }}/lightbulb
```

EXHIBIT A

Embedded parameter values and repetitive home directory value pattern in multiple places Works but could be more clearer and setup to be more flexible and maintainable

```
  key_file: /home/{{ username }}/.ssh/id_rsa
  repo: git@github.com:example/apprepo.git
```

SEPARATE LOGIC FROM VARIABLES

```yml
name: Clone student lesson app for a user
  host: nodes
vars:
user_home: /home/{{ username }}
user_ssh: "{{ user_home }}/.ssh"
deploy_key: "{{ user_ssh }}/id_rsa"
app_dest: "{{ user_home }}/exampleapp"
tasks:
name: Create ssh dir
      file:
state: directory
path: "{{ user_ssh }}"

name: Set Deployment Key copy:
src: files/deploy_key
dest: "{{ deploy_key }}"
```

EXHIBIT B

Parameters values are set thru values away from the task and can be overridden. Human meaningful variables “document” what’s getting plugged into a task parameter More easily refactored into a role

```
name: Clone repo
      git:
dest: "{{ app_dest }}"
key_file: "{{ deploy_key }}"
repo: git@github.com:example/exampleapp.git
```

PLAYS & TASKS

Use native YAML syntax to maximize the readability of your plays

- Vertical reading is easier
- Supports complex parameter values
- Works better with editor syntax highlighting in editors

USE NATIVE YAML SYNTAX

No!

```yaml
name: install telegraf
yum: name=telegraf-{{ telegraf_version }} state=present update_cache=yes disable_gpg_c notify: restart telegraf

name: configure telegraf
template: src=telegraf.conf.j2 dest=/etc/telegraf/telegraf.conf

name: start telegraf
service: name=telegraf state=started enabled=yes
```

USE NATIVE YAML SYNTAX

Better, but not quite all the way there...

```yaml
name: install telegraf
  yum: >
name=telegraf-{{ telegraf_version }}
state=present
update_cache=yes
disable_gpg_check=yes
enablerepo=telegraf
notify: restart telegraf

name: configure telegraf
template: src=telegraf.conf.j2 dest=/etc/telegraf/telegraf.conf

name: start telegraf
service: name=telegraf state=started enabled=yes
```

USE NATIVE YAML SYNTAX

```yaml
name: install telegraf
  yum:
  name: telegraf-{{ telegraf_version }} state: present
  update_cache: yes
  disable_gpg_check: yes
  enablerepo: telegraf
  notify: restart telegraf
template:
   src: telegraf.conf.j2
   dest: /etc/telegraf/telegraf.conf notify: restart telegraf
name: start telegraf
  service:
  name: telegraf
  state: started
  enabled: yes
```

PLAYs & TASKS

Names improve readability and user feedback

Give all your playbooks, tasks and blocks brief, reasonably unique and human-meaningful names

$myvar is never a good thing, and typing isn’t that hard

INVENTORY

```yml
- hosts: web
tasks:
- yum:
    name: httpd
    state: latest
- service:
    name: httpd
    state: started
    enabled: yes
```

```Shell
PLAY [web]
*********************************
TASK [setup]
********************************* ok: [web1]
TASK [yum]
********************************* ok: [web1]
  TASK [service]
********************************* ok: [web1]
```

PLAYS & TASKS

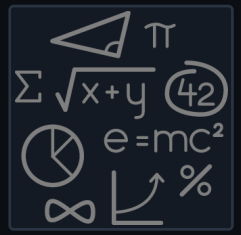
```yml
- hosts: web
- name: installs and start apache
tasks:
- name: install apache packages
  yum:
    name: httpd
    state: latest

- name: start apache service
  service:
    name: httpd
    state: started
    enabled: yes
```

```Shell
PLAY [install and start apache]
*********************************
TASK [setup]
********************************* ok: [web1]
TASK [install apache packages]
********************************* ok: [web1]
  TASK [start apache service]
********************************* ok: [web1]
```

PLAYS & TASKS

Focus avoids complexity



Keep plays and playbooks focused. Multiple simple playbooks are better than having a single, overburdened playbook full of conditional logic.

PLAYS & TASKS

Clean up your debugging tasks

Make them optional with the verbosity parameter so they’re only displayed when they are wanted.

```yaml
debug:
msg: "This always displays"

debug:
msg: "This only displays with ansible-playbook -vv+"
verbosity: 2
```

PLAYS & TASKS

Don’t just start services -- use smoke tests

```yaml
- name: check for proper response
  uri:
  url: http://localhost/myapp
  return_content: yes
  register: result
  until: '"Hello World" in result.content'
  retries: 10
  delay: 1
```

PLAYS & TASKS

Use command modules sparingly

- Use the run command modules like shell and command as a last resort
- Use the command module unless you really need I/O redirection that shell permits – but be very careful.

PLAY & TASKS

```yml
- name: add user
  command: useradd appuser
- name: install apache
  command: yum install httpd
- name: start apache
  shell: |
  service httpd start && chkconfig httpd on
```

```yml
- name: add user
  user:
    name: appuser
    state: present

- name: install apache yum:
    name: httpd
    state: latest

- name: start apache service:
    name: httpd
    state: started enabled: yes
```

PLAY & TASKS

Still using command modules a lot?

```yml
hosts: all
vars:
  cert_store: /etc/mycerts
  cert_name: my cert
tasks:
  name: check cert
  shell: certify --list --name={{ cert_name }} --
  cert_store={{ cert_store }} | grep "{{ cert_name }}"
  register: output

  name: create cert
  command: certify --create --user=chris --
  name={{ cert_name }} --cert_store={{ cert_store }}
  when: output.stdout.find(cert_name)" != -1
  register: output

  name: sign cert
  command: certify --sign --name={{ cert_name }} --
  cert_store={{ cert_store }}
  when: output.stdout.find("created")" != -1
```

PLAY & TASKS

Develop your own module! (seriously)

```yml
- hosts: all
  vars:
  cert_store: /etc/mycerts
  cert_name: my cert
tasks:
- name: create and sign cert
  certify:
    state: present
    sign: yes
    user: chris
    name: "{{ cert_name }}"
    cert_store: "{{ cert_store }}"
```

PLAY & TASKS

Separate provisioning from deployment and configuration tasks

```shell
acme_corp/
├── configure.yml
├── provision.yml
└── site.yml

$ cat site.yml
---
- import_playbook: provision.yml
- import_playbook: configure.yml
```

TEMPLATES

Jinja2 is powerful but you needn’t use all of it

Templates should be simple:

- Variable substitution
- Conditionals
- Simple control structures/iterations
- Design your templates for your use case, not the world’s

Things to avoid:

- Managing variables in a template
- Extensive and intricate conditionals
- Conditional logic based on embedded hostnames
- Complex nested iterations

TEMPLATES

Careful when mixing manual and automated configuration

Label template output files as being generated by Ansible

```yml
{{ ansible_managed | comment }}
```

ROLES

Roles are the shareable unit of work in Ansible

- Like playbooks – keep roles purpose and function focused
- Use a `roles/` subdirectory for roles developed for organizational clarity in a single project
- Follow the Ansible Galaxy pattern for roles that are to be shared beyond a single project
- Limit role dependencies

ROLES

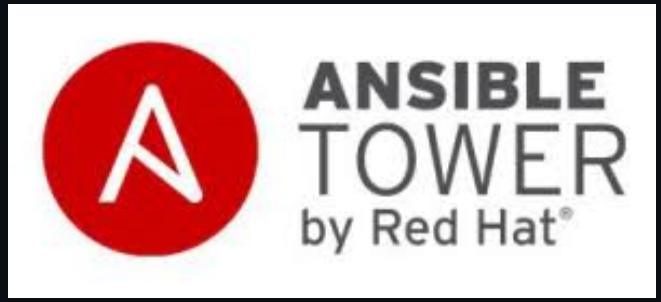
Sharing roles is paramount, and easy

- Use `ansible-galaxy init` to start your roles…
- …then remove unneeded directories and stub files
- Use `ansible-galaxy` to install your roles – even private ones
- Use a roles files (i.e. `requirements.yml`) to manifest any external roles your project is using
- Always specify a specific version such using a tag or commit for your roles

SCALING YOUR ANSIBLE WORKFLOW

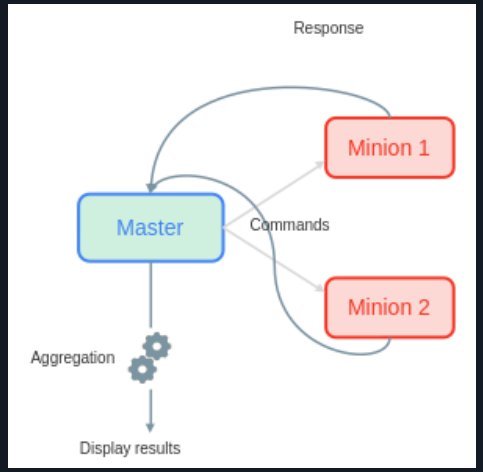
Command line tools have their limitations

- Coordination across a distributed teams & organization…
- Controlling access to credentials…
- Track, audit and report automation and management activity…
- Provide self-service or delegation…
- Integrate automation with enterprise systems…



Other solutions

Saltstack

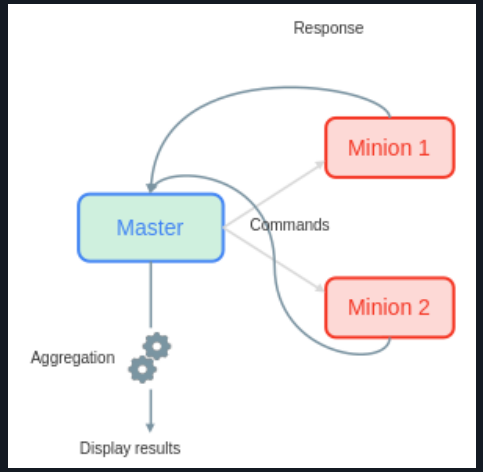


- Send commands via ZeroMQ or TCP
- Sending simultaneously to each minion
- Secure exchange via TLS
- Runs on Python on master and minion side,

but it’s possible to do without on minions by using a through a minion-proxy

- Everything is extensible/interchangeable, even the protocol

Saltstack



- Send commands via ZeroMQ or TCP
- Sending simultaneously to each minion
- Secure exchange via TLS
- Runs on Python on master and minion side,

but it’s possible to do without on minions by using a through a minion-proxy

- Everything is extensible/interchangeable, even the protocol

Saltstack - Installation

Master

```shell
sudo apt-get install salt-master
salt-master -d  # Start the service



salt-key -L  # List the keys
salt-key -a <id>  # Accept a key
```

Minion

```shell
sudo apt-get install salt-minion


sudo nano /etc/salt/minion
    master: master
    id: minion_01

salt-minion -d  # Start the service
```

Saltstack - States

```yml
nginx:
  pkg.installed:[]
  service.running:
  - require:
    - pkg: nginx
```

(1) DatasetID

(2) module.fonction

Install ‘nginx’ software with Saltstack

Saltstack - Execution

```
sudo salt '*' state.apply foo
```

Possibility of filtering minions other than by globs:

OS

```
salt -G 'os:Ubuntu'
```

Regex

```
salt -E 'minion[0-9]'
```

Enumeration

```
salt -L 'minion1,minion2'
```

Terraform/Tofu

Works through plugins (named Provider) that allow

interaction with the cloud services
Mainly designed to work with different cloud services cloud

services (AWT, Azure…)
Possibility to create its own providers in Go
Focuses mainly on provisioning of an infrastructure

Terraform/tofu - Installation

Terraform

```shell
sudo apt-get update && sudo apt-get install -y gnupg software-properties-common curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo apt-key add –
sudo apt-add-repository "deb [arch=amd64] https://apt.releases.hashicorp.com $(lsb_release -cs) main"
sudo apt-get update
sudo apt-get install terraform
```

OpenTofu

```shell
sudo apt update
sudo apt install snapd
sudo snap install opentofu --classic
```

Terraform/tofu - Configuration

```tf
terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
    }
  }
}

provider "docker" {}

resource "docker_image" "nginx" {
  name = "nginx:latest"
}


resource "docker_container" "nginx" {
  image = docker_image.nginx.latest
  name  = "nginx"
  ports {
    internal = 80
    external = 80
  }
}
```

Terraform/tofu - Execution

```shell
tofu init      } (1) Initialize a new directory (with new configuration)
tofu apply     } (2) Apply a configuration
tofu destroy   } (3) Destroy a configuration
```

Before a configuration is applied or deleted, a plan of the changes on the target machines is displayed by Terraform and must be validated

Terraform/tofu - Execution